

Lab Task 1

```
#include <iostream>

using namespace std;

int main()
{
    int n,m;
    char symbols[100];
    cin>>n>>m;

    for(int i=0;i<m;i++){
        cin>>symbols[i];
    }

    int transition[100][100];
    for(int i=0;i<100;i++){
        for(int j=0;j<100;j++){
            transition[i][j]=-1;
        }
    }

    int t;
    cin>>t;

    for (int i = 0; i < t; i++) {
        int currentState, nextState;

        char inputChar;

        cin >> currentState >> inputChar >> nextState;
```

```
int symbolIndex = -1;
```

```
for (int j = 0; j < m; j++) {  
    if (symbols[j] == inputChar) {  
        symbolIndex = j;  
        break;  
    }  
}
```

```
if (symbolIndex != -1) {  
    transition[currentState][symbolIndex] = nextState;  
}  
}
```

```
int startState;  
cin >> startState;
```

```
int f;  
cin >> f;
```

```
int finalStates[100];  
for (int i = 0; i < f; i++) {  
    cin >> finalStates[i];  
}
```

```
string inputString;  
cin >> inputString;
```

```
int currentState = startState;

bool valid = true;

for (int i = 0; i < inputString.length(); i++) {
    int symbolIndex = -1;

    for (int j = 0; j < m; j++) {
        if (symbols[j] == inputString[i]) {
            symbolIndex = j;
            break;
        }
    }

    if (symbolIndex == -1 || transition[currentState][symbolIndex] == -1) {
        valid = false;
        break;
    }

    currentState = transition[currentState][symbolIndex];
}

bool isFinal = false;
for (int i = 0; i < f; i++) {
    if (currentState == finalStates[i]) {
        isFinal = true;
        break;
    }
}
```

```
if (valid && isFinal) {  
    cout << "Accepted" << endl;  
} else {  
    cout << "Rejected" << endl;  
}  
  
return 0;  
}
```

```
3 2  
a  
b  
6  
0 a 1  
0 b 0  
1 a 1  
1 b 2  
2 a 1  
2 b 0  
0  
1  
2  
aab  
Accepted  
  
Process returned 0 (0x0)   execution time : 49.465 s  
Press any key to continue.
```

Lab Task 2

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
int main() {
```

```
    int n, m;
```

```
    cin >> n >> m;
```

```
    char symbols[100];
```

```
    for (int i = 0; i < m; i++) {
```

```
        cin >> symbols[i];
```

```
    }
```

```
    int transition[100][100];
```

```
    for (int i = 0; i < 100; i++) {
```

```
        for (int j = 0; j < 100; j++) {
```

```
            transition[i][j] = -1;
```

```
        }
```

```
    }
```

```
    int t;
```

```
    cin >> t;
```

```
    bool validDFA = true;
```

```
string reason = "";
```

```
for (int i = 0; i < t; i++) {
```

```
    int currentState, nextState;
```

```
    char inputChar;
```

```
    cin >> currentState >> inputChar >> nextState;
```

```
    if (currentState < 0 || currentState >= n || nextState < 0 || nextState >= n) {
```

```
        if (validDFA) {
```

```
            validDFA = false;
```

```
            reason = "State out of range";
```

```
        }
```

```
    }
```

```
int symbolIndex = -1;
```

```
for (int j = 0; j < m; j++) {
```

```
    if (symbols[j] == inputChar) {
```

```
        symbolIndex = j;
```

```
        break;
```

```
    }
```

```
}
```

```
if (symbolIndex == -1) {
```

```
    if (validDFA) {
```

```
        validDFA = false;
```

```
        reason = "Transition symbol not in alphabet";
```

```
    }
```

```
}
```

```

if (symbolIndex != -1) {
    if (transition[currentState][symbolIndex] != -1) {
        if (validDFA) {
            validDFA = false;
            reason = "Duplicate transition";
        }
    } else {
        transition[currentState][symbolIndex] = nextState;
    }
}
}

```

```

int startState;
cin >> startState;

```

```

if (startState < 0 || startState >= n) {
    if (validDFA) {
        validDFA = false;
        reason = "Invalid start state";
    }
}

```

```

int f;
cin >> f;

```

```

int finalStates[100];
for (int i = 0; i < f; i++) {
    cin >> finalStates[i];
}

```

```

    if (finalStates[i] < 0 || finalStates[i] >= n) {
        if (validDFA) {
            validDFA = false;
            reason = "Invalid final state";
        }
    }
}

```

```

int q;
cin >> q;

```

```

string testStrings[100];
for (int i = 0; i < q; i++) {
    cin >> testStrings[i];
}

```

```

for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        if (transition[i][j] == -1) {
            if (validDFA) {
                validDFA = false;
                reason = "Missing transition";
            }
        }
    }
}
}

```

```

if (!validDFA) {

```



```
    cout << "Invalid DFA" << endl;

    cout << reason << endl;

    return 0;
}
```

```
cout << "Valid DFA" << endl;

cout << endl;
```

```
for (int i = 0; i < q; i++) {
    string str = testStrings[i];

    int currentState = startState;

    bool rejected = false;

    cout << "String: " << str << endl;

    cout << "Path: " << currentState;
```

```
    for (int j = 0; j < str.length(); j++) {
        int symbolIndex = -1;
```

```
        for (int k = 0; k < m; k++) {
            if (symbols[k] == str[j]) {
                symbolIndex = k;
                break;
            }
        }
    }
```

```
    if (symbolIndex == -1 || transition[currentState][symbolIndex] == -1) {
        rejected = true;
        break;
    }
```

```
    }

    currentState = transition[currentState][symbolIndex];
    cout << " -> " << currentState;
}

cout << endl;

bool isFinal = false;
for (int j = 0; j < f; j++) {
    if (currentState == finalStates[j]) {
        isFinal = true;
        break;
    }
}

if (!rejected && isFinal) {
    cout << "Accepted" << endl;
} else {
    cout << "Rejected" << endl;
}

cout << endl;
}

return 0;
}
```

3 2

a

b

6

0 a 1

0 b 0

1 a 1

1 b 2

2 a 1

2 b 0

0

1

2

3

ab

aab

bbb

Valid DFA

String: ab

Path: 0 -> 1 -> 2

Accepted

String: aab

Path: 0 -> 1 -> 1 -> 2

Accepted

String: bbb

Path: 0 -> 0 -> 0 -> 0

Rejected

Process returned 0 (0x0) execution time : 58.746 s

Press any key to continue.